

Aula 11 - Autenticação JWT com Spring Security e Angular

Duração: 4 horas (1h teórica + 3h prática)

Parte Teórica (1 hora)

1. Revisão da Aula Anterior (15 min)

- Discussão sobre:
 - Validação de dados e tratamento de erros.
 - Dúvidas sobre `@ExceptionHandler` e DTOs de erro.

2. Introdução ao JWT (20 min)

O que é JWT?

- Token compacto e seguro no formato `Header.Payload.Signature`.
- Estrutura:
 - **Header:** Tipo do token + algoritmo de hash (ex: HS256).
 - **Payload:** Dados do usuário (ex: `sub`, `roles`, `exp`).
 - **Signature:** Verificação da integridade.

Fluxo de Autenticação

Diagram

Code

```
sequenceDiagram
    Client->>+Server: POST /login (credentials)
    Server-->>-Client: JWT
    Client->>+Server: GET /protected (Bearer JWT)
    Server-->>-Client: Dados protegidos
```

3. Spring Security Config (25 min)

Configuração Básica

- Dependências:

xml

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>

```

- **Classe de Configuração:**

```
java
```

```

@EnableWebSecurity
public class SecurityConfig {
    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf().disable()
            .authorizeRequests()
                .antMatchers("/api/auth/**").permitAll()
                .anyRequest().authenticated()
            .and()
            .addFilter(new JwtAuthFilter(authenticationManager()))
            .sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS);

        return http.build();
    }
}

```

Parte Prática (3 horas)

Atividade: Implementar Autenticação JWT

Passo 1: Backend - Login e Geração de Token (1h)

1. Classe `JwtUtil` (gera/valida tokens):

```
java
```

```

public class JwtUtil {
    private static final String SECRET_KEY = "chave-secreta-super-forte-123456";
    private static final long EXPIRATION_TIME = 86400000; // 1 dia

    public String generateToken(String username) {
        return Jwts.builder()

```

```

        .setSubject(username)
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + EXPIRATION_TIME))
        .signWith(SignatureAlgorithm.HS256, SECRET_KEY)
        .compact();
    }
}

```

2. Endpoint /api/auth/login :

java

```

@RestController
@RequestMapping("/api/auth")
public class AuthController {
    @Autowired
    private JwtUtil jwtUtil;

    @PostMapping("/login")
    public ResponseEntity<String> login(@RequestBody LoginRequest request) {
        // Valide credenciais (simplificado)
        if ("admin".equals(request.username()) && "123".equals(request.password())) {
            String token = jwtUtil.generateToken(request.username());
            return ResponseEntity.ok(token);
        }
        return ResponseEntity.status(401).build();
    }
}

```

Passo 2: Frontend - Serviço de Autenticação (1h)

1. Serviço Angular (auth.service.ts):

typescript

```

@Injectable({ providedIn: 'root' })
export class AuthService {
    private apiUrl = 'http://localhost:8080/api/auth';

    constructor(private http: HttpClient) {}

    login(credentials: { username: string; password: string }) {
        return this.http.post<{ token: string }>(`${this.apiUrl}/login`, credentials);
    }
}

```

2. Componente de Login (login.component.ts):

typescript

```

export class LoginComponent {
  credentials = { username: '', password: '' };

  constructor(private authService: AuthService, private router: Router) {}

  onSubmit() {
    this.authService.login(this.credentials).subscribe({
      next: (res) => {
        localStorage.setItem('token', res.token);
        this.router.navigate(['/']);
      },
      error: (err) => alert('Login falhou!')
    });
  }
}

```

Passo 3: Proteção de Rotas (1h)

1. Interceptor para adicionar token (jwt.interceptor.ts):

typescript

```

@Injectable()
export class JwtInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler) {
    const token = localStorage.getItem('token');
    if (token) {
      req = req.clone({
        setHeaders: { Authorization: `Bearer ${token}` }
      });
    }
    return next.handle(req);
  }
}

```

2. Guarda de Rotas (auth.guard.ts):

typescript

```

@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private router: Router) {}

  canActivate(): boolean {
    if (!localStorage.getItem('token')) {
      this.router.navigate(['/login']);
      return false;
    }
    return true;
  }
}

```

```
}  
}
```

3. Aplicar guarda nas rotas (`app-routing.module.ts`):

typescript

```
const routes: Routes = [  
  { path: 'login', component: LoginComponent },  
  { path: '', component: HomeComponent, canActivate: [AuthGuard] }  
];
```

Tarefa para Casa

1. **Refresh Token:** Implementar renovação automática do token.
2. **Proteção no Backend:** Criar um `JwtAuthFilter` para validar tokens nas requisições.

Próxima Aula

- **Tópico:** Upload de Arquivos + Paginação.
- **Atividade prática:** Sistema de perfil de usuário com foto.

Resultado Esperado

- Sistema de login funcional com:
 - Geração/validação de JWT no backend.
 - Armazenamento de token no frontend.
 - Rotas protegidas em ambos os lados.

plaintext

- ✓ Login/logout funcionais
- ✓ Rotas protegidas
- ✓ Token incluído automaticamente nas requisições

https://miro.medium.com/v2/resize:fit:1400/1*5X_uNqi_UV1IJq6L9Q2UIQ.png (Fluxo JWT entre Angular e Spring Boot) 🚀